Task W12: Heaps

Top Priority

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Overview

This task has three required problems and one optional challenge problem. Problems 1 and 2 are the same kind of problem we work through in lecture: a sequence of insertions and deletions, first on a minheap and then on a maxheap. Problem 3 traces heapsort on an array. The optional challenge problem at the end asks you to decide whether an array holds a heap, which is a check on the definition itself.
Use the heap algorithms from our lectures and notes.

- **Insert:** place the new item as the last node, then trickle it up.
- **Delete:** remove the root, place the last node as the root, then trickle it down. When trickling down, always move toward the child with the **higher priority** (the smaller child in a minheap, the larger child in a maxheap).

2 Diagram Submission Instructions

Problems 1, 2, and 3 ask for diagrams. You may choose one of the following options.

- **Hand-drawn on paper or a whiteboard.** Scan or take clear, well-lit photos. Images must be easy to read and clearly labeled with problem numbers. For this task in particular, this is the recommended option: heaps are much faster to sketch by hand than to build in software.
- **Hand-drawn using an app** (for example Vittle Pro or Adobe Fresco). Save as an image file (PNG, JPG) or PDF. This is a convenient option but may be harder without a stylus.
- **Created with diagramming software** (such as Draw.io, LibreOffice Draw, or Excalidraw). Save in the tool's native file format; do not export as an image or PDF.
- **Other tools.** If you would like to use a different diagramming application, ask for approval first. Note that I have limited installation access on my office computer.

Include your name and the problem number on every diagram.

If you find something that needs to be debugged

If, while working through this assignment, you discover something in the problem statements that needs to be debugged, describe it in the space at the end of the answer sheet. This is separate from, and in addition to, your answers.

Where to submit

Type your answers in the answer sheet that comes with this task (a Word or OpenDocument file), and save it as .docx or .odt. Put everything for this task (your answer sheet and your diagram file) in a folder named `taskw12` in your GitHub repository, and commit it.

3 Conventions

These match the notes.

- A heap is stored in an array with the root at **index 0**. For the item at index i, the parent is at $(i-1)/2$ (integer division), the left child is at $2i+1$, and the right child is at $2i+2$.
- Some keys appear more than once. An item never moves past an item with an equal key: a trickle up stops below an equal parent, and a trickle down stops above an equal child. If both children have the same key, move toward the **left** child.
- A **delete** always removes the root, because the root holds the highest priority.

4 Problem 1: A Minheap

Start with an **empty minheap** and perform the twelve operations in Table 1, in order.

Table 1: The twelve operations for Problems 1 and 2. A delete removes the root.

Step	Operation
1	Insert 58
2	Insert 70
3	Insert 58
4	Insert 33
5	Insert 70
6	Insert 81
7	Delete
8	Insert 41
9	Insert 81
10	Insert 59
11	Delete
12	Delete

Draw the heap after every operation, twelve drawings in all, and label each one with its step number. Each drawing should show the finished heap for that step. You may also show the swaps along the way, but you do not have to.

5 Problem 2: A Maxheap

Start with an **empty maxheap** and perform the same twelve operations from Table 1, in order.

(a) **Draw the heap after every operation,** twelve drawings in all, labeled with their step numbers, as in Problem 1.

(b) Your final heaps from Problems 1 and 2 have the same **shape** (the same nodes in the same positions) but different **contents**. Explain why the shape is the same even though the two heaps followed different rules the whole way through.

6 Problem 3: Heapsort

Use heapsort to sort the array in Table 2 into increasing order. Use the algorithm from the notes: phase 1 builds a maxheap, and phase 2 repeatedly swaps the root with the last item of the heap and trickles the new root down.

Table 2: The array for Problem 3, with its indexes.

Index	0	1	2	3	4	5	6	7
Item	36	60	45	53	15	35	10	62

(a) **Phase 1.** At which index does phase 1 start, and why does it start there instead of at the last index? Then, on the answer sheet, write the array after each trickle down in phase 1, one row per index.

(b) **Draw the maxheap** that phase 1 produces, as a tree.

(c) **Phase 2.** On the answer sheet, write the array after each swap and trickle down, one row per step. In each row, put a vertical bar (|) between the heap part and the sorted part of the array.

7 Challenge Problem (Optional): Is This a Heap?

This problem is optional. It is not required, and it does not earn extra credit. It is here for anyone who wants to check their understanding of the definition of a heap.
Each row of Table 3 is an array. The word "empty" means that slot of the array holds no item.

Table 3: Three arrays for the challenge problem. Each row lists the items at indexes 0 through 7. In array C, index 4 is empty.

Array	0	1	2	3	4	5	6	7
A	90	72	85	40	65	80	30	45
B	4	9	15	22	31	38	47	50
C	95	70	88	52	empty	61	40	33

For each array:

(a) Say whether it is a valid **maxheap**, a valid **minheap**, or **neither**.

(b) If it is neither, say which kind of heap it comes closest to, and list every place it fails as that kind. Give each failure as a pair of indexes (a parent and a child that are out of order) or as the index of an empty slot.

(c) In one or two sentences, explain your answer.

You may find it helpful to sketch each array as a tree, but you do not need to submit those sketches. All of the challenge problem is typed on the answer sheet.

8 What to Submit

Typed answers go in the answer sheet, and drawings go in your diagram file.

- **Problem 1:** twelve drawings.
- **Problem 2:** twelve drawings and your explanation for (b).
- **Problem 3:** the starting index and its explanation, the phase 1 and phase 2 arrays, and one drawing of the heap after phase 1.
- **Challenge problem (optional):** typed answers for arrays A, B, and C.
- Anything that needs to be debugged, in the space at the end of the answer sheet.

Everything above goes in the `taskw12` folder in your GitHub repository.